

File Systems in Linux

Concepts and Commands

A. Mustafi

January 15, 2023

1 Introduction

Essentially everything in Unix is considered as a file. This is an overriding philosophy and one which has no exceptions. While files are storehouses of information, there are three types of files in UNIX. These are

Ordinary Files - These are files which hold information usually provided by the user.

Directory Files - These files hold information about other files in a subdirectory.

Device Files - These files hold information about physical devices that make up the UNIX system.

One very important aspect about files is the fact that a file in UNIX shall not contain anything except the contents of the file. This is in contrast to MS-Windows/MS-DOS where a file contains metadata as well as control information like EOF etc. Filenames in modern UNIX systems can be of virtually any length though many systems limit to a maximum of 255 characters and can contain any character except the `"/`. There is no concept of a file extension, and it is up to the user or a particular compiler to decide if an extension is required.

UNIX file systems follow a parent child hierarchy with the *root* (`/`) directory at the top. Some common subdirectories of root are:

`/bin` and `/usr/bin` - usually contains executables and therefore houses most UNIX commands.

`/sbin` and `/usr/sbin` - commands specifically available to system admins and not to normal users.

`/etc` - contains the configuration files for the system.

`/dev` - contains all device files.

/home and /home//usr/ - this folder houses all users with individual directories for each user created as subdirectories.

/tmp - all temporary files are stored here.

/var - holds all variable length data e.g. print queues, incoming mails etc.

/lib - contains all library files.

2 Commands for working with the file system

2.1 Introducing ls

ls is a command to see the directory listing of any directory. We shall be doing a lot more on ls on in future classes.

2.2 Finding out your base location

The two commands that help us out in this regard are *pwd* which expands to present working directory and *echo \$HOME* which shows the home directory for the current logged in user.

2.3 Absolute and relative pathnames

You can specify a position in the file system in context of the root of the system or with respect to your current location. The former is known as an *absolute path* while the latter is known as a *relative path*. We can specify the location of the current folder using the *.* i.e. *single dot* while the parent folder can always be specified using *..* i.e. *double dot*.

2.4 Some commands for basic file system handling

Table 1: File System commands

Command	Effect	Syntax	Multiple args.
cd	Change to a directory	cd <i><pathname></i>	No
mkdir	Create a new directory	mkdir <i><pathname></i>	Yes
rmdir	Deletes a directory	rmdir <i><pathname></i>	Yes
cp	copies a file	cp <i><pathname(s)><folder></i>	Yes
rm	deletes a file	cp <i><pathname(s)><folder></i>	Yes
mv	moves or renames a file	mv <i><pathname(s)><folder></i>	Yes
cat	creates a new file or displays a file	cat <i><pathname(s)></i>	No
file	know a file type	file <i><pathname(s)></i>	Yes
df	find out free space	cp df <i><pathname(s)></i>	No
du	User diskspace consumption	du <i><pathname(s)></i>	No

Many of the commands listed in Table 1 make use of *metacharacters* and take options. Please see the man pages for details.

2.5 Compressing files

Basic file compression is performed using the command *compress* and its sibling *uncompress*. The compression offered is only moderate and *compress* does delete the original file.

More modern users tend to prefer *gzip* with the decompressor being *gunzip*. *gzip* tends to offer slightly less compression than *compress* but is tremendously efficient.

You also have the choice of using *zip* which can compress multiple files into a single archive.

3 Assignments

1. Create four directories in your home directory called `dir1`, `dir2`, `dir3`, and `dir4`.
2. Move to `dir1` using an absolute pathname.
3. Create two files called `file1` and `file2` with some content in each.
4. Copy `file1` to `dir2` using absolute pathnames.
5. Copy both `file1` and `file2` to `dir3` using relative path and metacharacters.
6. Copy `file1` from `dir2` to `dir3` using an interactive form of the copy command.
7. Copy both files from `dir1` to `dir4` and rename them to `file41` and `file42`.
8. How can you see contents of both files `file41` and `file42` together?
9. Remove both files `file41` and `file42` in an interactive manner.
10. Return to your home directory and try to run the command `mkdir /dir5/dir51 /dir5`. Why does this not work?
11. While in your home directory try to remove the directory `dir4`. What do you observe?
12. Create three files called `*`, `file55`, `file56` in `dir5` with some content. How will you see content of the file called `.` only.
13. Try to delete the file named `.` in `dir5`.
14. Create a zip archive in `dir6` (which you need to create) containing all the files in `dir1`.
15. Create a subdirectory in `dir6` called `dir61`. Also create two text files in `dir6` called `file61` and `file62` with some content. How can you copy the entire contents of `dir6` to its parent directory?